TECHNISCHE HOCHSCHULE OSTWESTFALEN-LIPPE, UNIVERSITY OF APPLIED SCIENCES AND ARTS

DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

GAMIFICATION INNOVATION LAB - WINTER TERM 2025

Interactive 3D Campus Environments for Gamification Using Gaussian Splatting in Unreal Engine

Nithya Kanakam(20139023)

Keerthana Kothakapu Adamulla(20141455)

Jahnavi Kalla(20165318)

Venkata Karthik Bommidi(20141413)

Serin Sara Samuel(20172716)

CONTENTS

I. Abstract 3

II. Introduction 3

2.1 Problem Statement 4
2.2 Task Description 4

III. State of the art 5

3.1 Evolution of Image-Based 3D Reconstruction Methods 5
3.2 Tools and Technologies Used 5
3.3 System Requirements 6

IV. Methodology 6

4.1 Data Collection 6
4.2 Images to Gaussian Splats 7
4.3 Visualization and Refinement with SuperSplat 9
4.4 Integration into Unreal Engine 9

V. Results 10

5.1 Overview of Results 10
5.2 Quantitative Results from Pipeline 1 10
5.3 Qualitative Results from Pipeline 1 10
5.4 Results from Pipeline 2: DGX Server-Based Automated Workflow 11
5.5 Unreal Engine Integration and Interactive Result 12

VI. Discussion 13

6.1 Data Acquisition Limitations 13
6.2 Training and Hardware Constraints 14
6.3 Integration and Deployment Limitations 14
6.4 Overall Observations 14

VII. Conclusion 15

7.1 Future Scope 15

References 17

Appendix 17

I. ABSTRACT

This project investigates the use of 3D Gaussian Splatting for reconstructing a real-world university campus environment and integrating it into an interactive game-engine-based application. The objective is to transform multi-view image data of the Innovation Spin building at TH OWL into a real-time navigable 3D scene. Unlike traditional photogrammetry and Neural Radiance Field (NeRF) methods, 3D Gaussian Splatting represents scenes using optimized Gaussian primitives, enabling high-quality reconstruction with efficient GPU-based rendering suitable for real-time applications. The pipeline involves image-based data acquisition, Structure-from-Motion processing for camera pose estimation, Gaussian Splat training, post-processing, and integration into Unreal Engine for interactive visualization. Approximately 300 GB of campus image data was collected using a GoPro camera, from which a 60 GB subset was selected due to computational constraints. The resulting reconstructed model was refined and exported in PLY format. It was then imported into Unreal Engine to enable real-time exploration. The results demonstrate that Gaussian Splatting is an effective approach for generating visually realistic and interactive 3D environments from real-world data. However, challenges such as high memory requirements, large dataset handling, and limited automation highlight areas for future improvement. Overall, the study confirms the potential of Gaussian Splatting for applications in educational environments, digital twins, and gamified spatial systems.

II. INTRODUCTION

Gamification refers to the integration of game design elements such as interaction, rewards, and challenges into non-game contexts to improve user engagement, motivation, and learning outcomes. It is widely used in education, training simulations, and virtual environments, where immersion and active participation are important. Recent research highlights the growing importance of 3D digital learning environments, where spatial representation improves interaction and overall user experience [1].

To support these systems, high-quality 3D environments based on real-world data are required. Traditional 3D modeling techniques often involve significant manual effort and are not efficient for large or complex environments. As a result, research has increasingly focused on automated methods that can reconstruct real-world scenes while supporting interactive applications.

One major development in this field is Neural Radiance Fields (NeRF), introduced by Mildenhall et al. [2]. NeRF represents scenes using neural networks trained on images to map spatial coordinates and viewing directions to color and density values. It can produce highly realistic images from new viewpoints, especially for effects like lighting and reflections. However, it requires multiple neural network computations for each ray, which makes rendering slow and computationally expensive, limiting its use in real-time systems.

To overcome these limitations, Gaussian Splatting, introduced by Kerbl et al. [3], models scenes using 3D Gaussian elements defined by position, color, opacity, and covariance (which defines shape and orientation). Rendering is performed using fast GPU-based techniques, allowing real-time performance while maintaining high visual quality. Unlike NeRF, this approach avoids expensive ray-based computations and is more suitable for interactive applications.

Recent studies show that Gaussian Splatting performs better than NeRF-based methods in terms of speed, stability, and scalability, especially in complex or limited-view scenarios [4]. These advantages make it a strong choice for applications that require both realism and efficiency.

These developments are highly relevant for modern game engines such as Unreal Engine, which are used to build virtual reality (VR), augmented reality (AR), and simulation environments. Such platforms rely on realistic 3D content to create immersive experiences. In this project, Gaussian Splatting is used to reconstruct parts of a real-world university campus and integrate them into a game engine, demonstrating how real-world data can be transformed into interactive environments that run in real time.

2.1 Problem Statement

The aim of this project is to reconstruct parts of a university campus from images and video data using Gaussian Splatting and to import the resulting 3D models into Unreal Engine for real-time visualization and interaction. The project also evaluates how suitable this approach is for interactive applications based on rendering speed, visual quality, and usability within a game engine.

2.2 Task Description

Gaussian Splatting Pipeline Workflow

- **Data Collection**
  Images and video frames of the campus are captured from multiple viewpoints, ensuring enough overlap for accurate reconstruction.
- **COLMAP Processing (Structure-from-Motion)**
  The images are processed using Structure-from-Motion (SfM) to estimate camera positions, orientations, and intrinsic parameters, producing a basic 3D representation made of scattered points.
- **Gaussian Initialization**
  The point cloud is converted into Gaussian elements, each defined by position, color, opacity, and shape.
- **Training and Optimization**
  The scene is rendered and compared with the input images, and the model is repeatedly adjusted to improve reconstruction accuracy.
- **Adaptive Refinement**
  Gaussian elements are dynamically adjusted by adding, splitting, or removing them to improve detail in complex areas and reduce unnecessary data in simpler regions.
- **Real-Time Rendering**
  Gaussian elements are projected onto the screen and rendered using fast GPU-based methods for real-time performance.
- **Model Export (PLY Format)**
  The final model is exported as a PLY file containing the optimized 3D representation.
- **Unreal Engine Integration**
  The model is imported into Unreal Engine, where it can be visualized, navigated, and interacted with in real-time.

III. STATE OF THE ART

3.1 Evolution of Image-Based 3D Reconstruction Methods

Image-based reconstruction aims to recover both the shape and appearance of a scene from multiple images. Traditional approaches use photogrammetry techniques such as Structure-from-Motion (SfM) and Multi-View Stereo (MVS) to estimate camera parameters and reconstruct 3D geometry. These methods produce point clouds or mesh models and are widely used. However, they have difficulty capturing complex effects such as reflections, transparency, and changing lighting conditions, which limits realism in rendered views [5].

Neural rendering methods were developed to address these limitations, with Neural Radiance Fields (NeRF) being a key advancement [2],[6]. NeRF models scenes using neural networks that learn how light behaves in a scene. It produces highly realistic results by sampling rays through the scene to compute pixel colors. However, because it requires many computations for each ray, it is slow and not suitable for real-time applications [6].

More recently, 3D Gaussian Splatting has emerged as a state-of-the-art method for image-based reconstruction [3]. Instead of using neural networks for every rendering step, it represents scenes using Gaussian elements, which can be rendered efficiently using rasterization. This approach significantly reduces computation while maintaining high visual quality.

A key strength of Gaussian Splatting is its ability to adjust the number and distribution of Gaussian elements during training, improving both detail and efficiency. Studies show that it converges faster and performs better in complex scenes compared to both photogrammetry and NeRF-based approaches [7].

Overall, the development from photogrammetry to neural rendering and then to Gaussian-based methods shows a clear shift toward solutions that balance realism, speed, and scalability. Based on current research, Gaussian Splatting is considered one of the most effective approaches for real-time 3D reconstruction.

3.2 Tools and Technologies Used

3.2.1 Data Collection and Structure-from-Motion (SfM)

- **GoPro Camera**
  Used to capture high-resolution images and videos from multiple viewpoints.
- **COLMAP**
  Used for Structure-from-Motion to estimate camera parameters and generate a sparse 3D point cloud.
- **RealityScan**
  Used to automate photogrammetry processing and generates COLMAP-compatible outputs.

3.2.2 Gaussian Splat Training

- **Graphdeco Gaussian Splatting Framework**
  Used to train and optimize Gaussian models of the scene [11].
- **LichtFeld Studio**
  Provides a graphical interface for training, parameter tuning, and real-time visualization [10].

3.2.3 Visualization, Refinement, and Game Engine Integration

- **SuperSplat**
  Used to visualize and clean the model by removing noise and unnecessary elements.
- **Unreal Engine 5.1.1 with X3DGVerse Plugin**
  Used for real-time rendering and interaction. The plugin allows direct import of Gaussian Splat (.ply) files[8],[9].

3.2.4 Toolchain Summary

This project uses a complete pipeline that combines photogrammetry tools, Gaussian Splatting frameworks, visualization tools, and a game engine. Together, these components convert real-world image data into optimized 3D models that can be explored interactively in real time.

3.3 System Requirements

3.3.1 Local Device

- **Device:** 2023 HP OMEN 16 (Model 16-xf0060AX)
- **Processor & Memory:** AMD Ryzen 7 CPU with 16 GB DDR5 RAM (5600 MHz)
- **Graphics:** NVIDIA GeForce RTX 4060 GPU with 8 GB GDDR6 VRAM

3.3.2 Server Environment

- **Server:** NVIDIA DGX A100 AI Server (TH OWL)
- **Environment:** Docker-based GPU-enabled setup (PyTorch / Jupyter Notebook)
- **Compute Resources:** High-performance multi-GPU system shared among users

IV.METHODOLOGY

4.1 Data Collection

The data collection process involved capturing images of the campus environment to generate a 3D reconstruction using Gaussian Splatting. The images were recorded using a GoPro camera with the guidance and assistance of the project supervisor.

To ensure comprehensive coverage of the campus, images were taken from multiple angles and positions while moving through different areas. The camera was typically handheld or mounted and moved steadily through the environment to simulate a continuous scanning process. A key requirement for Gaussian Splatting is **approximately 80% overlap** between consecutive images. Therefore, care was taken to capture the same structures from slightly different viewpoints, which helps the reconstruction algorithm accurately estimate camera positions and align the images in 3D space.

In addition, AprilTags (fiducial markers) were placed at various locations throughout the campus during the data collection phase to support image acquisition. These markers were intended to provide identifiable reference points within the environment. However, the dataset used in this project consisted of preprocessed images in which the AprilTags were not visibly present. Therefore, they were not directly utilized in the reconstruction pipeline implemented in this work.

The captured images were stored directly on the SD card and later transferred to a computer for processing. Due to the large scale of the campus, the data collection process took several weeks to complete, resulting in a total dataset of approximately 300 GB of image data.

However, due to hardware and computational constraints, processing the entire dataset was not feasible within the scope of this project. Therefore, a representative subset of approximately 60 GB of data, focusing on the **Innovation Spin building**, was selected for Gaussian Splatting reconstruction. This subset still provided sufficient detail and coverage for generating a high-quality 3D representation of the selected area.

4.2 Images to Gaussian Splats

To convert the collected images into Gaussian Splatting representations, two different pipelines were implemented and evaluated.

Pipeline 1: RealityScan + LichtFeld Studio

In the first pipeline, the captured images were processed using RealityScan. The tool internally performs reconstruction using COLMAP-based techniques, meaning that camera calibration and scene reconstruction are handled automatically.

As part of this process, RealityScan generates COLMAP outputs, including:

- **cameras.txt:** Contains intrinsic camera parameters such as camera model, image resolution, focal length, and principal point.
- **images.txt:** Defines camera poses (rotation and translation), indicating the position of each image in 3D space.
- **points3D.txt:** Represents the sparse point cloud with 3D coordinates, colour information, and reconstruction error values.

Additionally, undistorted images are generated to improve reconstruction accuracy.

These outputs were then imported into LichtFeld Studio, where Gaussian Splatting training was performed. Using the estimated camera parameters and images, the system generated a Gaussian-based scene representation, which was exported as a .ply file.

The key training parameters used in LichtFeld Studio were as follows:

- Iterations: 30,000
- Maximum Gaussians: 2,000,000
- SH Degree: 3
- Tile Mode: 1

Pros

- Provides strong graphical user interface (GUI) control and real-time feedback, enabling intuitive parameter tuning.
- Automated reconstruction workflow reduces manual setup effort.

Cons

- Limited scalability due to GPU memory constraints (typically ~2–4 million Gaussians).
- Low flexibility for automation, making it unsuitable for large-scale or batch processing pipelines.

Pipeline 2: Graphdeco Gaussian Splatting (Headless Training on DGX Server)

The second pipeline was implemented on a DGX server via SSH access without a graphical user interface, enabling the use of high-performance GPU resources. The official Gaussian Splatting implementation from the INRIA Graphdeco group was used for this process [11].

In this approach, reconstruction and training were carried out in two main stages. Initially, the environment was configured by installing the required dependencies, including COLMAP for Structure-from-Motion and camera pose estimation.

In the first stage, the convert.py script was used to perform feature extraction, image matching, and camera parameter estimation, resulting in a sparse 3D reconstruction of the scene.

In the second stage, the train.py script was executed to train the Gaussian Splatting model using the outputs from the previous step. This stage optimizes a set of 3D Gaussians to represent the scene and produces the final reconstruction as a .ply file.

To streamline the workflow, a shell script was used to automate the entire pipeline. This script sequentially executes the preprocessing and training steps, producing the final output without manual intervention.

The training process allows control over parameters such as the number of iterations, which directly affects model convergence and reconstruction quality. The pipeline was executed in a headless environment, meaning that monitoring was performed through command-line logs rather than real-time visualization.

Pros

- High computational performance using DGX GPU resources.
- Scalable and automated workflow suitable for larger datasets.

Cons

- No graphical interface, making debugging and visualization more difficult.
- The pipeline offers less intuitive parameter control compared to GUI-based approaches.
- Limited real-time feedback, as training progress is observable only through logs.

Pipeline Comparison and Selection

Both pipelines were evaluated for generating Gaussian Splat models of the campus environment. After comparing the outputs in terms of visual quality and reconstruction stability, the model that produced the most visually consistent and high-quality result was selected.

The selected output was used for further refinement and integration into Unreal Engine, as it provided the most reliable reconstruction for the project.

4.3 Visualization and Refinement using SuperSplat

Following the generation of the initial .ply outputs from the Gaussian Splatting pipeline, a post-processing step was performed using SuperSplat to refine and prepare the data for further use.

The generated model was first imported into SuperSplat, which provides an interactive environment for inspecting and editing Gaussian Splat representations. This enabled direct visualization of the reconstructed scene, allowing qualitative evaluation of reconstruction quality and comparison between outputs from different pipelines.

A key aspect of this stage was the removal of noise and unwanted Gaussian splats. During training, the reconstruction process may introduce artifacts such as floating points, redundant splats, or poorly optimized regions. These were manually or semi-automatically cleaned within SuperSplat to improve the overall clarity and structural consistency of the model.

After refinement, the optimized model was exported again in .ply format. While the file format remains the same, the processed output is cleaner and better suited for real-time rendering in downstream applications such as Unreal Engine.

4.4 Integration into Unreal Engine

The refined .ply model was imported into Unreal Engine 5.1.1 to serve as the primary 3D environment for the developed application. The engine was installed via the Epic Games Launcher, and the project was configured with the X3DGS plugin to enable support for Gaussian Splat-based .ply files.

Once imported, the reconstructed scene was rendered as a fully navigable environment within the engine. Using the built-in first-person template, an interactive application was developed that allows users to freely move through and explore the reconstructed environment.

To enhance interactivity, simple gameplay elements were introduced. Interactive objects were placed within the scene, allowing user engagement through basic interaction mechanics. Specifically, targetable blocks were implemented, which can be interacted with using a shooting-based interaction system.

This stage represents the final step of the pipeline, where reconstructed real-world data is transformed into an interactive virtual environment. It demonstrates the practical application of Gaussian Splatting in a real-time system, enabling immersive exploration and interaction within reconstructed scenes

V. RESULTS

5.1 Overview of Results

This section presents the results obtained from the Gaussian Splatting reconstruction and Unreal Engine integration of the TH OWL campus environment. The project results are based on two pipelines. Pipeline 1 used RealityScan and LichtFeld Studio with approximately 60 GB of InnovationSPIN campus data. Pipeline 2 used the Graphdeco Gaussian Splatting framework on the DGX server with a smaller 6 GB Detmold campus dataset due to memory limitations. The final objective of both pipelines was to generate Gaussian Splat .ply files that could be exported and used as real-time 3D environments in Unreal Engine.

5.2 Quantitative Results from Pipeline 1

Pipeline 1 was completed using RealityScan for reconstruction preparation and LichtFeld Studio for Gaussian Splat training. The training process was completed successfully with the following metrics:

Metric	**Result**
Training duration	14 h 37 min 47 s
Training iterations	30,000
Final loss	0.0438
Final GPU memory usage	8.6 / 8.6 GB

Maximum Gaussians	2,000,000
SH degree	3
Tile mode	1

Table.1: Quantitative Results from Pipeline 1

5.3 Qualitative Results from Pipeline 1

The generated Gaussian Splat model achieved a high level of visual fidelity as shown in Fig. 1 and Fig. 2., especially for the main campus building. The overall structure of the InnovationSPIN building was reconstructed clearly, including the building geometry, window placement, façade surfaces, and surrounding campus elements. Lighting reflections and material appearance were also preserved well, which contributed to a realistic visual impression.

The reconstruction was particularly effective for the primary objects that were well covered during image capture. The building façade, entrance areas, paved surfaces, vegetation, and nearby architectural elements remained recognizable and visually consistent from different viewpoints. This confirms that Gaussian Splatting is suitable for producing realistic campus-scale visualizations when sufficient image overlap and viewpoint.coverage are available.

*Fig.*1: Gaussian splats of innovation spin building

*Fig.*2: Gaussian splats of innovation spin building

5.4 Results from Pipeline 2: DGX Server-Based Automated Workflow

Pipeline 2 was implemented as an automated headless workflow using the Graphdeco Gaussian Splatting framework on the university DGX server. The aim of this pipeline was to test a more scalable and script-based reconstruction process that could reduce manual interaction and support future large-scale campus processing.

Initially, the intention was to process a larger dataset similar to Pipeline 1. However, due to memory limitations on the available server environment, the full dataset could not be processed successfully. Therefore, a smaller dataset of approximately 6 GB, captured from the TH OWL Detmold campus, was selected and executed through the automated pipeline as shown in Fig. 3.

This pipeline was executed using a custom script that automates the main processing stages, including image preparation, COLMAP-based conversion, Gaussian Splat training, and final .ply output generation. Compared with Pipeline 1, which used approximately 60 GB of data and relied on RealityScan and LichtFeld Studio, Pipeline 2 focused more on automation, reproducibility, and command-line execution..

The images generated from Pipeline 2 will be included in the report to show the visual outcome of the automated workflow. These results are useful for validating the pipeline structure and identifying future improvements, especially in terms of memory management, dataset preparation, and scalability.

Fig.3: Gaussian splats of Detmold campus building

Since Pipeline 1 and Pipeline 2 were applied to different datasets and campus areas, their results should not be interpreted as direct visual or quantitative comparisons. Instead, they demonstrate complementary strengths: Pipeline 1 produced a high-quality reconstruction of the InnovationSPIN building, while Pipeline 2 demonstrated the feasibility of an automated server-based reconstruction workflow on a smaller dataset.

5.5 Unreal Engine Integration and Interactive Result

The refined Gaussian Splat model was imported into Unreal Engine using the X3DGS plugin as shown in Fig. 4, as described in the project workflow. This was one of the most important results of the project because it transformed the reconstructed campus model from a static visualization into an interactive real-time environment.

Inside Unreal Engine, the Innovation SPIN campus model was used as the base environment for a small gamified campus tour. A first-person view was implemented, allowing the user to move through and explore the reconstructed campus area. Basic interactive game elements, including blocks and a rifle asset, were added to test interaction inside the environment. This demonstrated that Gaussian Splat-based campus reconstructions can be used not only for viewing, but also as a foundation for game-like experiences.

The Unreal Engine result shows that the developed pipeline can support future gamified applications such as campus tours, educational exploration, sustainability challenges, and interactive learning modules. Although the current gamification is limited, it proves that additional gameplay mechanics can be built on top of the reconstructed Gaussian Splat environment.

Overall, the project achieved three important results. First, Pipeline 1 successfully reconstructed the InnovationSPIN building from a 60 GB dataset with strong visual quality. Second, SuperSplat post-processing improved the model by reducing noise and preparing it for real-time use. Third, the refined model was successfully integrated into Unreal Engine, where a first-person interactive campus tour with basic game elements was created.

These results confirm that the developed workflow can serve as a foundation for future campus-based games, digital tours, and gamified learning applications.

Fig.4: Campus interactive environment in unreal engine

VI. DISCUSSION

6.1 Data Acquisition Limitations

The implementation of the Gaussian Splatting pipeline demonstrated strong potential for reconstructing real-world environments and integrating them into an interactive game engine. However, several practical challenges were encountered throughout data collection, training, and system deployment.

A major challenge was related to lighting conditions during data acquisition. Ideally, image capture for 3D reconstruction should be performed under uniform lighting conditions with minimal shadows and consistent exposure. However, since the data collection took place during winter, lighting conditions were not always ideal. This resulted in variations such as shadows, low-light regions, and uneven exposure across images. These inconsistencies introduced additional challenges during reconstruction, particularly affecting feature matching and the quality of some reconstructed regions.

Furthermore, the data acquisition process was constrained by practical limitations. Due to institutional restrictions, permission to use drone-based data collection was not granted. Consequently, all images had to be captured manually using a GoPro camera. While this approach was feasible, it required significant effort to ensure sufficient coverage and overlap between viewpoints. Maintaining consistent image quality and spatial coverage during manual capture was challenging, particularly in larger or complex areas of the environment.

6.2 Training and Hardware Constraints

Another key limitation was the availability of computational resources on the TH OWL AI (DGX) server. Although the server provides high-performance GPU capabilities, access was often restricted due to high usage by multiple users. As a result, only a limited portion of GPU memory (approximately 30–50 GB availability at a given time) could be reliably accessed for training. For Gaussian Splatting, higher memory capacity is beneficial for handling dense reconstructions, and these resource constraints directly affected the scale and efficiency of training.

An important observation emerged during the evaluation of the second pipeline implemented on the DGX server. Using a sample dataset from the Detmold campus (approximately 6 GB), the Gaussian Splatting training required significantly higher GPU memory than was readily available on the system, with an estimated requirement of around 80 GB for full-resolution training. Due to these limitations, the input resolution was reduced by a factor of 4 to decrease memory consumption and make the training feasible under the available resources. After applying this optimization, the training process could be completed on the DGX server, thereby successfully implementing Pipeline 2 under constrained computational conditions.

This experiment highlighted the strong dependency of Gaussian Splatting on GPU memory when processing high-density datasets. Extrapolating from this behavior, larger datasets such as the Innovation Spin building dataset (approximately 60 GB) would likely require substantially higher computational resources, estimated to be around 120 GB of GPU memory, depending on scene complexity and reconstruction density.

As a result of these scalability constraints, the first pipeline (RealityScan combined with LichtFeld Studio) was adopted as the primary reconstruction approach for the Innovation Spin dataset. This pipeline provided a more resource-efficient workflow, enabling stable training and manageable memory usage while still producing a high-quality reconstruction suitable for integration into Unreal Engine.

6.3 Integration and Deployment Limitations

An additional limitation was encountered during the integration of the generated .ply models into Unreal Engine. When importing the Gaussian Splatting output into Unreal Engine 5.1.1, a noticeable reduction in visual quality was observed compared to the original rendered output. This is primarily due to differences in rendering pipelines and the limitations of the available plugin used for Gaussian Splat visualization. Furthermore, the project was constrained to Unreal Engine 5.1.1, as newer engine versions do not provide stable or supported .ply import functionality for Gaussian Splatting data. As a result, the integration process was limited to a compatible engine version, which restricted access to newer rendering improvements and optimization features.

6.4 Overall Observations

Despite these limitations, the combination of pipelines including RealityScan-based processing and the Graphdeco Gaussian Splatting framework enabled successful reconstruction of the selected area. The DGX server, although frequently busy, was utilized effectively whenever available, and training was completed through careful resource management and optimization strategies.

Overall, the results demonstrate that Gaussian Splatting is highly effective for real-time 3D reconstruction. However, its practical implementation is strongly influenced by factors such as hardware availability, data quality, and acquisition constraints. Through adaptive strategies and multiple pipeline approaches, a complete end-to-end reconstruction workflow was successfully achieved under restricted conditions.

VII.CONCLUSION

This project demonstrates that 3D Gaussian Splatting can effectively transform real-world image data into interactive, real-time 3D environments within Unreal Engine. By designing and evaluating two reconstruction pipelines one GUI-based and one scalable server-based a flexible workflow was established for generating Gaussian Splat models from campus data. Despite practical constraints related to lighting, hardware, and data acquisition, a high-quality reconstruction was successfully achieved and integrated into a game engine. While no full game system was developed, the resulting environment provides a solid foundation for future gamified applications such as virtual campus exploration and educational simulations. Overall, this work highlights Gaussian Splatting as a practical and efficient approach for bridging real-world data capture with interactive digital environments.

7.1 Future Scope — Gamified Campus Digital Twin

From Static Model to Interactive Experience

In the future, the reconstructed campus can be extended into a gamified digital twin as shown in Fig. 4. Instead of only allowing users to view the campus, the system can provide interactive challenges, learning tasks, and decision-based activities. This would transform the Gaussian Splat environment into an educational and exploratory platform for students, visitors, and prospective students.

Sustainability Module

A sustainability-based game mode can allow users to plant and maintain trees, place solar panels on suitable building areas, and find efficient walking routes across campus. These actions can contribute to a Sustainability Score, encouraging users to understand environmental planning and sustainable campus development.

Efficiency Module

An efficiency module can focus on energy and resource optimization. Campus buildings could be visually marked from inefficient to efficient, and users could adjust parameters such as lighting, heating, or building usage. Based on these choices, the system can provide visual feedback and calculate an Efficiency Score.

Knowledge Module

A knowledge module can introduce interactive hotspots across the campus. These hotspots can provide information about facilities, laboratories, research areas, and SmartFactoryOWL projects. Quiz-based interactions can help users test their understanding and earn a Knowledge Score.

Exploration and Guided Tour Layer

The campus environment can also include free navigation, building hotspots, hidden locations, and guided tour paths. This would make the application useful not only as a technical prototype, but also as a digital campus tour and marketing platform for TH OWL.

Long-Term Vision

The long-term goal is to develop the project into a complete Campus Intelligence system where users can explore, learn, optimize, and interact with a realistic digital version of the university. By combining Gaussian Splatting with gamification, the campus model can become more than a visual reconstruction: it can become an immersive learning and decision-making environment.

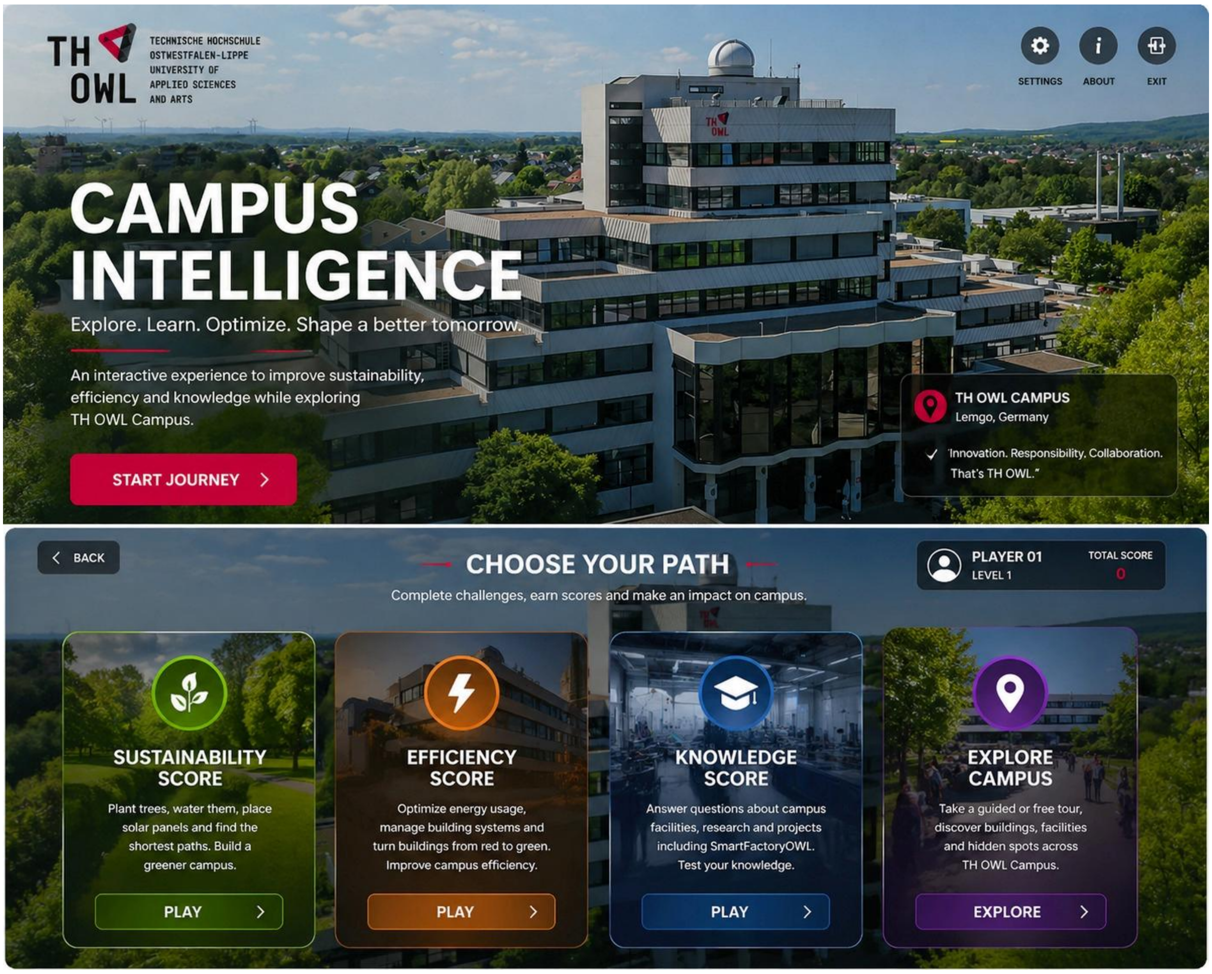

Fig.5: Gamified Campus Digital Twin story board

REFERENCES

[1] S. Das, S. V. Nakshatram, H. Söbke, J. B. Hauge, and C. Springer, "Towards gamification for spatial digital learning environments," Entertainment Computing, 2024, doi: 10.1016/j.entcom.2024.100893.

[2] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," European Conference on Computer Vision (ECCV), 2020.

[3] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," ACM Transactions on Graphics, vol. 42, no. 4, 2023.

[4] C. Blanchard, L. Gupta, and S. Nanisetty, "Analyzing 3D Gaussian Splatting and Neural Radiance Fields: A Comparative Study on Complex Scenes and Sparse Views," 2024.

[5]A. Javadi Moghadam, A. Kiani, R. Naeimaei, S. Malihi, and I. Brilakis, "Video-Based 3D Reconstruction: A Review of Photogrammetry and Visual SLAM Approaches," *Journal of Imaging*, vol. 12, no. 3, p. 128, Mar. 2026.

[6] K. Gao, Y. Gao, H. He, D. Lu, L. Xu, and J. Li, "NeRF: Neural Radiance Field in 3D Vision, A Comprehensive Review," arXiv preprint arXiv:2210.00379, 2022.

[7] M. Carmeliti et al., "A Comparative Evaluation of 3D Reconstruction with Photogrammetry, NeRFs and 3D Gaussian Splatting in Cultural Heritage Restoration," The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, vol. XLVIII-2/W12-2026, pp. 89–96, 2026.

[8] X. Jin and X. Han, "UE-GS: High-Quality 3D Gaussian Splatting Real-Time Rendering for Unreal Engine," in Proc. 7th International Conference on Advanced Algorithms and Control Engineering, 2024.

[9] YHK-UEPlugins-Public, "UE Gaussian Splatting Plugin," GitHub repository. Available: https://github.com/YHK-UEPlugins-Public/018_UEGaussianSplatting_Public

[10] MrNeRF, "LichtFeld Studio," GitHub repository. Available: https://github.com/MrNeRF/LichtFeld-Studio

[11] GraphDeco-INRIA, "Gaussian Splatting," GitHub repository. Available: https://github.com/graphdeco-inria/gaussian-splatting

APPENDIX

The Bash script gaussian_splatting.sh was used to automate Docker setup, COLMAP preprocessing, model training, and final .ply export. The script is included in the submitted archive.

Additionally, the generated Gaussian Splatting .ply files of the reconstructed campus environment are available at the following link for reference and visualization:

https://drive.google.com/drive/folders/1_tO3gzsEIkANluHxq5s5XHTrmE8MaNiY?usp=sharing